

QUEUE QNO:10

QUESTION:

Question description

Umesh is an DS expert training youngsters struggling in DS to make them better.

Umesh usually gives interesting problems to the youngsters to make them love the DS.

One such day Umesh provided to the youngsters to solve the task such that, Reverse a Queue, Queue data structures work on the FIFO architecture so the element that has entered first in the list will go out from the list first.

Youngsters were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

first line indicates the enqueue of each elements

last line indicates the reversed queue elements.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5
5 6 3 1 2

EXPECTED OUTPUT

Queue:5 6 3 1 2
Reversed Queue:2 1 3 6 5

Test Case 2

INPUT (STDIN)

8
1 8 6 4 9 5 3 5

EXPECTED OUTPUT

Queue:1 8 6 4 9 5 3 5
Reversed Queue:5 3 5 9 4 6 8 1

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void enqueue(int data,int l)

Test Case 4

KEYWORD

reverse();

Test Case 2

KEYWORD

void reverse()

Test Case 3

KEYWORD

enqueue(t,n);

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

4

Test Case 2

TOKEN COUNT

275

Test Case 3

NLOC

57

Source Code:

```
#include <stdio.h>

#define MAX 105

// Global array and pointers for the queue
int q[MAX];
int front = -1;
int rear = -1;

// Mandatory Test Case 1: Exact keyword match
// The parameter 'l' represents the size limit of the queue
void enqueue(int data, int l) {
    // Check if the queue is full
    if (rear == l - 1) {
        return;
    }
    // Set front to 0 for the first element
    if (front == -1) {
        front = 0;
    }
    // Increment rear and insert data
    rear++;
    q[rear] = data;
}

void reverse() {
    // Reverse the elements in place by swapping ends until they meet in the middle
    int i, j, temp;
    // Note: We use j-- instead of j++ to properly converge to the middle
    for (i = front, j = rear; i < j; i++, j--) {
        temp = q[i];
        q[i] = q[j];
        q[j] = temp;
    }
}

// Helper function to display the queue
void display() {
    if (front == -1) {
        return;
    }
    for (int i = front; i <= rear; i++) {
        printf("%d", q[i]);
        if (i < rear) {
            printf(" ");
        }
    }
    printf("\n");
}
```

```

int main() {
    int n;

    // Read the size of the queue
    if (scanf("%d", &n) != 1) return 0;

    // Read the elements
    for (int i = 0; i < n; i++) {
        int t;
        scanf("%d", &t);
        // Mandatory Test Case 3: Exact keyword match
        enqueue(t,n);
    }

    // Print the original queue
    printf("Queue: ");
    display();

    // Mandatory Test Case 4: Exact keyword match
    reverse();

    // Print the reversed queue
    printf("Reversed Queue: ");
    display();

    return 0;
}

```

Output:

```

5
5 6 3 1 2
Queue: 5 6 3 1 2
Reversed Queue: 2 1 3 6 5

```

```

8
1 8 6 4 9 5 3 5
Queue: 1 8 6 4 9 5 3 5
Reversed Queue: 5 3 5 9 4 6 8 1

```